

# README

Course: **Operating Systems [CS2012E]**

Assignment: **Assignment 2** - *Verifying if the given number is perfect with Multithreading*

**Submitted by:**

Name: **Kona Bhuvaneswar Reddy**

Roll No: **B240214CS**

Class: **CS02**

Date: **10th April, 2026**

## Description

This program is a multithreaded solution implemented in **C** to determine if a given integer **N** is a **Perfect Number**. A perfect number is defined as a positive integer that is equal to the sum of its proper divisors (excluding itself).

This program utilizes the **POSIX Threads (pthreads)** library to parallelize the search for factors. To ensure high performance, it employs the  $O(\sqrt{N})$  optimization and minimizes lock contention by using local thread accumulator.

## Environment Requirements

- **Operating System:** Linux/Unix-based environment (or Windows with WSL/MinGW).
- **Compiler:** **gcc** (GNU Compiler Collection).
- **Libraries:** **pthread** (threading) and **m** (math).

## Compilation Steps

Since the program uses the thread and math libraries, you must explicitly link them during the compilation process.

Open your terminal and run:

**Bash:** `gcc B240214CS_A2.c -o perfect_check -lpthread -lm`

- **-o perfect\_check:** Names the output executable "perfect\_check".
- **-lpthread:** Links the POSIX thread library.
- **-lm:** Links the math library (required for **sqrt**).

## Execution Instructions

The program requires two command-line arguments: the number to check (**N**) and the number of threads to spawn (**P**).

**Syntax:**

**Bash:** `./perfect_check <N> <P>`

## Examples:

- **Checking a perfect number (28) with 4 threads:**

**Bash:** `./perfect 28 4`

**Output:** `true`

- **Checking a non-perfect number (10) with 2 threads:**

**Bash:** `./perfect 10 2`

**Output:** `false`

- **Checking a large perfect number (8128) with 8 threads:**

**Bash:** `./perfect 8128 8`

**Output:** `true`

## Implementation Details

- **Workload Partitioning:** The search space  $[1, \sqrt{N}]$  is divided into  $P$  roughly equal segments.
- **Synchronization:** A `pthread_mutex_t` is used to protect the shared `total_sum` variable from race conditions.
- **Optimization:** Each thread calculates a `local_sum` for its assigned range and only locks the mutex **once** at the end of its execution to update the global total. This prevents threads from constantly blocking each other.

## Troubleshooting

- **Permission Denied:** If you cannot execute the file, run `chmod +x perfect`.
- **Math/Thread Errors:** Ensure you included `-lm` and `-lpthread` at the end of your `gcc` command.
- **Windows Users:** It is highly recommended to run this in **VS Code using the WSL (Ubuntu) extension** or a Linux terminal to ensure compatibility with the `pthread.h` header.